

PRINCIPIOS SOLID EN EL DESARROLLO DE SOFTWARE MODERNO

PRESENTADO POR:

Miguel Ángel Herrera

PRESENTADO A:

Diana María Valencia

UNIVERSIDAD ALEXANDER VON HUMBOLDT

FACULTAD DE INGENIERÍAS

PROGRAMA DE INGENIERÍA DE SOFTWARE

Armenia/Quindio

08/06/2026

1. Introducción

En el campo del desarrollo de software, la calidad del código no se mide únicamente por su funcionalidad, sino también por su capacidad de mantenerse, evolucionar y escalar con el tiempo. A medida que los sistemas crecen en complejidad, se hace evidente la necesidad de principios que guíen la toma de decisiones durante el diseño y la implementación.

Los principios SOLID, formulados y recopilados por Robert C. Martin a principios de la década de 2000, representan un conjunto de cinco directrices fundamentales para el diseño orientado a objetos. Su nombre es un acrónimo formado por las iniciales de cada principio: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation y Dependency Inversion. Aunque cada uno tiene un enfoque particular, en conjunto conforman una base sólida para construir software robusto, comprensible y fácil de mantener.

2. Los Cinco Principios SOLID

2.1 S — Principio de Responsabilidad Única (Single Responsibility Principle)

Definición

Una clase debe tener una única razón para cambiar, lo que equivale a decir que debe ocuparse de una sola responsabilidad dentro del sistema. Cuando una clase asume múltiples responsabilidades, los cambios en una de ellas pueden afectar involuntariamente a las demás, aumentando la probabilidad de errores y dificultando el mantenimiento.

Ejemplo práctico

Una clase llamada Factura que, además de calcular el total de una venta, también se encarga de imprimir el documento y guardarlo en la base de datos. Esta clase tiene tres responsabilidades distintas. Aplicando el SRP, se separaría en tres clases independientes: Factura (lógica de negocio), ImpresoraFactura (presentación) y RepositorioFactura (persistencia). Así, un cambio en el formato de impresión no afectaría la lógica de cálculo ni el acceso a datos.

2.2 O — Principio de Abierto/Cerrado (Open/Closed Principle)

Definición

Las entidades de software como clases, módulos o funciones deben estar abiertas para su extensión, pero cerradas para su modificación. Esto significa que el comportamiento de una clase puede ampliarse sin necesidad de alterar su código fuente existente, lo cual reduce el riesgo de introducir bugs en funcionalidad ya probada.

Ejemplo práctico

En un sistema de procesamiento de pagos que inicialmente solo admite tarjetas de crédito. Si se diseña con una interfaz MedioPago y una clase TarjetaCredito que la implementa, al necesitar agregar pagos en efectivo o por transferencia bancaria, simplemente se crean nuevas clases (Efectivo, Transferencia) que implementan la misma interfaz. La clase procesadora de pagos no requiere modificarse, solo extenderse mediante nuevas implementaciones.

2.3 L — Principio de Sustitución de Liskov (Liskov Substitution Principle)

Definición

Formulado por Barbara Liskov en 1987, este principio establece que los objetos de una subclase deben poder sustituir a los de su clase base sin alterar el comportamiento correcto del programa. En términos prácticos, una subclase no debe eliminar ni restringir el comportamiento prometido por la clase padre, ya que esto rompe las expectativas de quien usa la abstracción.

Ejemplo práctico

Se tienen las clases Rectangulo y Cuadrado. Si Cuadrado hereda de Rectangulo y sobrescribe el método setAncho() para que también modifique el alto, se viola el LSP, porque el código que usa un Rectangulo esperaría poder cambiar ancho y alto de forma independiente. La solución correcta es no establecer esa jerarquía o diseñar una abstracción más general (FiguraGeometrica) de la cual ambas clases hereden sin comprometer sus contratos.

2.4 I — Principio de Segregación de Interfaces (Interface Segregation Principle)

Definición

Ningún cliente debería verse obligado a depender de métodos que no utiliza. Este principio promueve la creación de interfaces pequeñas y específicas en lugar de interfaces generales con muchos métodos. Cuando una interfaz es demasiado amplia, las clases que la implementan se ven forzadas a proporcionar implementaciones vacías o innecesarias, lo que genera acoplamiento innecesario.

Ejemplo práctico

Una interfaz `Trabajador` con métodos `trabajar()`, `comer()` y `dormir()`. Si se necesita implementar un `RobotTrabajador`, este tendría que implementar `comer()` y `dormir()`, lo cual carece de sentido. Aplicando el ISP, se dividirían las interfaces en `Trabajable` (`trabajar()`), `Comedor` (`comer()`) y `Dormidor` (`dormir()`), de forma que cada clase implemente únicamente lo que le corresponde.

2.5 D — Principio de Inversión de Dependencias (Dependency Inversion Principle)

Definición

Este principio tiene dos enunciados complementarios: primero, los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones. Segundo, las abstracciones no deben depender de los detalles; son los detalles quienes deben depender de las abstracciones. Esto invierte la dirección tradicional de dependencia y reduce el acoplamiento entre componentes.

Ejemplo práctico

Una clase `ServicioNotificaciones` que instancia directamente un objeto `EnvioEmail` está acoplada a esa tecnología concreta. Si en el futuro se necesita enviar notificaciones por SMS o push, habría que modificar la clase. Aplicando el DIP, `ServicioNotificaciones` dependería de una interfaz `INotificador`, y sería la capa externa quien inyecte la implementación concreta (`EnvioEmail`, `EnvioSMS`, etc.). Esto facilita enormemente el intercambio de componentes y las pruebas unitarias.

3. Ventajas de la Implementación de SOLID

3.1 Mantenibilidad

Uno de los beneficios más evidentes de aplicar SOLID es la mejora en la mantenibilidad del código. Cuando cada clase tiene una única responsabilidad y las dependencias están bien definidas a través de abstracciones, localizar y corregir errores se vuelve considerablemente más sencillo. Los desarrolladores pueden intervenir en una parte del sistema sin preocuparse por generar efectos no deseados en otras. Además, el código bien estructurado resulta más fácil de leer y entender, lo que reduce el tiempo de incorporación de nuevos miembros al equipo y disminuye la deuda técnica acumulada.

3.2 Escalabilidad

Un sistema diseñado con los principios SOLID escala de manera más ordenada. Al estar los componentes desacoplados y organizados alrededor de abstracciones, añadir nuevas funcionalidades no implica reestructurar el código existente. Esto permite que el sistema crezca de forma incremental sin comprometer su estabilidad. La inversión de dependencias, en particular, facilita la distribución de responsabilidades entre módulos independientes que pueden evolucionar de forma paralela, lo cual es crítico en equipos de desarrollo grandes o en arquitecturas de microservicios.

3.3 Extensibilidad

El principio Abierto/Cerrado, junto con el de Segregación de Interfaces, sienta las bases para una arquitectura altamente extensible. Agregar nuevas funcionalidades se logra creando nuevas clases o implementaciones, sin tocar el código base existente. Esto no solo reduce el riesgo de introducir regresiones, sino que también permite evolucionar el software de acuerdo con los cambios del negocio o la tecnología de forma ágil y controlada.

3.4 Testabilidad

Aunque no se menciona explícitamente en el enunciado del acrónimo, la testabilidad es una consecuencia directa de aplicar estos principios. El bajo acoplamiento y la dependencia en abstracciones hacen posible el uso de objetos simulados (mocks) y la escritura de pruebas unitarias aisladas. Esto, a su vez, favorece la adopción de metodologías como el desarrollo guiado por pruebas (TDD), fortaleciendo la calidad general del producto.

4. Reflexión Personal

De los cinco principios, el que más me llamó la atención fue el Principio de Inversión de Dependencias. La idea central es que el código de alto nivel no debería estar atado a los detalles de implementación, sino a abstracciones. Eso cambia la forma en que uno piensa el diseño: en lugar de preguntar "¿cómo lo hago funcionar?", te obliga a preguntarte "¿de qué debería depender esto?".

Un sistema que depende de abstracciones y no de implementaciones concretas es mucho más fácil de adaptar cuando algo cambia, y en software, algo siempre termina cambiando.

Lo que más me parece interesante es que el DIP tiene un efecto en cadena sobre el diseño general: cuando las dependencias están bien invertidas, casi naturalmente se empieza a respetar la responsabilidad única, a diseñar interfaces más limpias y a mantener jerarquías coherentes.

Entender este principio también cambia la manera en que se lee el código. Uno empieza a notar cuándo una clase está demasiado acoplada, cuándo un cambio en un módulo rompe cosas que no deberían verse afectadas, y esa capacidad de identificar el problema es el primer paso para escribir mejor software.

Link video en drive:

https://drive.google.com/drive/folders/1hSLHeX79bQR0tb9mNpAXGqA_-4sXv5az?usp=sharing

5. Referencias

Martin, R. C. (2003). *Agile Software Development, Principles, Patterns, and Practices*. Pearson Education.

Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

Liskov, B. H., & Wing, J. M. (1994). A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6), 1811–1841.

Freeman, E., & Robson, E. (2004). *Head First Design Patterns*. O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.